# SKKU 2024 Challenge - Quantum State Tomography

Donghun Jung

November 8, 2024

## 1 Convention

## Problem 1: Quantum States with Few Qubits

The first class of states that we will consider are generic pure quantum states (i.e. there is no restriction on the quantum circuit which created the state), on a small number of qubits. In general, a generic quantum state will require an exponential number of measurements to reconstruct. However, when the number of qubits is small, we can simply pay this exponential cost.

### Part I

In this first problem, we have a single qubit quantum state

$$|\psi\rangle = a\,|0\rangle + be^{i\theta}\,|1\rangle\,, \tag{1}$$

where we can treat the unknown parameters $a, b$ and $\theta$ as real positive numbers with $|a|^2 + |b|^2 = 1$ and $\theta \in [0, 2\pi)$.

**a) Explain how one can determine the values of $a, b$ and $\theta$ by measuring the quantum state in different bases.**

Starting with the single-qubit pure state:

$$|\psi\rangle = a\,|0\rangle + be^{i\theta}\,|1\rangle \tag{2}$$

where $a, b, \theta$ are real parameter subject to $a^2 + b^2 = 1$ and $0 \leq \theta < 2\pi$. To determine these parameters, we need at least three distinct measurements. We measure the state using POVMs in different bases, and by adding unitary operations before measurement, we can perform each measurement in a modified basis.

**First Measurement(Z-basis)** In this scenerio, we perform $z$-basis projection measurement. Therefore, the POVM becomes $\{|0\rangle\langle 0|, |1\rangle\langle 1|\}$, each corresponds to the presense and absence of the signal. Let $\hat{m}_0, \hat{m}_1$ denote the estimated probability of occuring signals. We can add additional operation $U$ before the measurement and after the computational stage(here, state preparation), we can perform measurement another POVM basis, $\left\{U_{\text{add}}^{\dagger}|0\rangle\langle 0|U_{\text{add}}, U_{\text{add}}^{\dagger}|1\rangle\langle 1|U_{\text{add}}\right\}$. As we should run several circuits, let $U_{\text{add}^i}$ denote the additional circuit to $i$-th circuit. In first measurement, to determine $a, b$, we can directly perform measurement without any unitary operation. That is $U_{\text{add}}^1 = I$, here. Then, we see:

$$\hat{m}_0 \simeq |\langle\psi|0\rangle|^2 = a^2 \tag{3}$$

$$\hat{m}_1 \simeq |\langle\psi|1\rangle|^2 = b^2 \tag{4}$$

Thus, we estimate,

$$a = \sqrt{\hat{m}_0} \tag{5}$$

$$b = \sqrt{\hat{m}_1} \tag{6}$$

**Second Measurement(X-basis)** Apply a Hadamard gate, $U_{\text{add}}^2 = H$, where $H$ is a hadamard gate.

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{7}$$

Now the measurement basis becomes $\{|+\rangle\langle +|, |-\rangle\langle -|\}$ where $|+\rangle, |-\rangle$ are as the eigenstates of the Pauli-X matrix. Here, let $\hat{m}_+, \hat{m}_-$ be the estimated probability of occuring signals.

$$\hat{m}_+ \simeq |\langle\psi|+\rangle|^2 = \left|\frac{a + be^{i\theta}}{2}\right|^2 = \frac{1 + 2ab\cos\theta}{2} \tag{8}$$

$$\hat{m}_- \simeq |\langle\psi|-\rangle|^2 = \left|\frac{a - be^{i\theta}}{2}\right|^2 = \frac{1 - 2ab\cos\theta}{2} \tag{9}$$

Or, Solving for $\cos\theta$

$$\cos\theta = \frac{\hat{m}_+ - \hat{m}_-}{2ab} \tag{10}$$

**Third Measurement(Y-basis)** Apply the S gate, $U_{\text{add}^3} = SH$ where

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}. \tag{11}$$

This creates a measurement basis $\{|i\rangle\langle i|, |-i\rangle\langle -i|\}$ where $|i\rangle, |-i\rangle$ are the eigenstate of Pauli-Y matrix. Here, let $\hat{m}_i, \hat{m}_{-i}$ be the estimated probability

of occuring signals.

$$\hat{m}_i \simeq |\langle\psi|i\rangle|^2 = \left|\frac{a + be^{i(\theta+\frac{\pi}{2})}}{2}\right|^2 = \frac{1 - 2ab\sin\theta}{2} \tag{12}$$

$$\hat{m}_{-i} \simeq |\langle\psi|-i\rangle|^2 = \left|\frac{a - be^{i\theta+\frac{\pi}{2}}}{2}\right|^2 = \frac{1 + 2ab\sin\theta}{2} \tag{13}$$

Or, Solving for $\sin\theta$

$$\sin\theta = \frac{-\hat{m}_i + \hat{m}_{-i}}{2ab}. \tag{14}$$

Finally, combining these results gives: $a, b$ and $\theta$.

$$a = \sqrt{\hat{m}_0} \tag{15}$$

$$b = \sqrt{\hat{m}_1} \tag{16}$$

$$\theta = \arcsin\frac{-\hat{m}_i + \hat{m}_{-i}}{2ab} = \arccos\frac{\hat{m}_+ - \hat{m}_-}{2ab} \tag{17}$$

**b) Write a program in Qiskit which generates this unknown quantum state. To generate a random single qubit quantum state we use a circuit $U_{\text{hidden}}$, so that $|\psi\rangle = U_{\text{hidden}}|0\rangle$. I used the following provided code, to generate random state.**

```
def create_1q_target_circuit(qc_data):
    qc = qiskit.QuantumCircuit(1)
    qc.ry(qc_data[0],0)
    qc.rz(qc_data[1],0)
    return qc
```

As discussed in part (a), we can determine the state in Eq. 2 by performing three measurements. The code below implements this state reconstruction process.

```
def SingleQubitQST(U_hidden, num_shots:int=100):
  hidden_circuit = U_hidden.copy()

  qc1 = qiskit.QuantumCircuit(1)
  circ1 = hidden_circuit.compose(qc1)
  circ1.measure_all()
  #print(circ1.draw())

  qc2 = qiskit.QuantumCircuit(1)
```

```
qc2.h(0)
circ2 = hidden_circuit.compose(qc2)
circ2.measure_all()
#print(circ2.draw())

qc3 = qiskit.QuantumCircuit(1)
qc3.s(0)
qc3.h(0)
circ3 = hidden_circuit.compose(qc3)
circ3.measure_all()
#print(circ3.draw())

result1 = MeasurementResultHandler(backend
        .run(circ1, shots=num_shots)
        .result().data()["counts"], 1, num_shots)
result2 = MeasurementResultHandler(backend
        .run(circ2, shots=num_shots)
        .result().data()["counts"], 1, num_shots)
result3 = MeasurementResultHandler(backend
        .run(circ3, shots=num_shots)
        .result().data()["counts"], 1, num_shots)

m0 = result1.get("0x0"); m1 = result1.get("0x1")
if m0 == 1:
  return np.array([1,0], dtype=np.complex64)
elif m1 == 1:
  return np.array([0,1], dtype=np.complex64)

phi = np.arctan2(np.sqrt(m1), np.sqrt(m0))
phi = phi if phi > 0 else phi + 2*np.pi
statevector = np.array([np.cos(phi),
                        np.sin(phi)],
                        dtype=np.complex64)

m_p = result2.get("0x0"); m_m = result2.get("0x1")
cos = (m_p - m_m)/(2*np.cos(phi)*np.sin(phi))

m_pi = result3.get("0x0"); m_mi = result3.get("0x1")
sin = (-m_pi + m_mi)/(2*np.cos(phi)*np.sin(phi))

temp = np.array([cos, sin])

cos = cos / np.linalg.norm(temp)
```

```
  sin = sin / np.linalg.norm(temp)
  print(circ1.draw())
  theta = np.arctan2(sin, cos)
  theta = theta if theta > 0 else theta + 2*np.pi

  statevector[1] *= np.exp(1j*theta)
  circuit_parameter = np.array([2*phi, theta])

  return statevector, circuit_parameter
```

With the reconstructed state vector, we then determine the circuit parameters for generating $U_{\text{gen}}$. Using ZYZ decomposition, we can express any single-qubit unitary operation. For state preparation, only RY and RZ gate[2, 3, 1]. This prepares the state as:

$$|\psi\rangle = R_z(\beta)R_y(\alpha) = \cos\left(\frac{\alpha}{2}\right)|0\rangle + e^{i\beta}\sin\left(\frac{\alpha}{2}\right)|1\rangle\,. \tag{18}$$

without global phase. By comparing with Eq. 2, we find the circuit parameters $\alpha, \beta$.

$$\alpha = 2\arccos a = 2\arcsin b \tag{19}$$

$$\beta = \theta \tag{20}$$

The following function computes these parameters and constructs $U_{\text{gen}}$.

```
def SingleQubitCircuit(statevector):
  cos = np.linalg.norm(statevector[0])
  sin = np.linalg.norm(statevector[1])
  phi = np.arctan2(sin,cos)
  theta = (np.log(statevector[1]/
          np.linalg.norm(statevector[1]))*-1j).real
            if statevector[1] != 0 else 0

  U_gen = qiskit.QuantumCircuit(1)
  U_gen.ry(2*phi,0)
  U_gen.rz(theta,0)

  return U_gen
```

To evaluate the fidelity, we use the following code:

```
backend2 = Aer.get_backend("statevector_simulator")
def give_score(U_hidden, U_gen):
  qc = U_hidden.compose(U_gen.inverse())
  result = backend2.run(qc).result()
  score = result.get_statevector()[0]
  return np.abs(score)
```

## Part II

We will now increase the complexity by reconstructing a generic two-qubit quantum state.

$$|\psi\rangle = a\,|00\rangle + b\,|01\rangle + c\,|10\rangle + |11\rangle \tag{21}$$

where $|a|^2 + |b|^2 + |c|^2 + |d|^2 = 1$.

**a) First, consider the case where all amplitudes are real numbers. Explain how you can measure both the magnitude and sign of the unknown amplitudes, measuring in as few different bases as possible.**

If the parameters are real numbers, three circuits are sufficient to determine all parameters $a, b, c$ and $d$.

In the first circuit, we measure the amplitudes directly with $U^0_{\text{add}} = I_4$:

$$\hat{m}_{00} \simeq |\langle\psi|00\rangle|^2 = a^2 \tag{22}$$
$$\hat{m}_{01} \simeq |\langle\psi|01\rangle|^2 = b^2 \tag{23}$$
$$\hat{m}_{10} \simeq |\langle\psi|10\rangle|^2 = c^2 \tag{24}$$
$$\hat{m}_{11} \simeq |\langle\psi|11\rangle|^2 = d^2 \tag{25}$$

From these measurements:

$$a = \sqrt{m_{00}} \tag{26}$$
$$b = \sqrt{m_{01}} \tag{27}$$
$$c = \sqrt{m_{10}} \tag{28}$$
$$d = \sqrt{m_{11}} \tag{29}$$
$$\tag{30}$$

Next, we apply a Hadamard gate to the second qubit with $U^1_{\text{add}} = I \otimes H$

and measure:

$$\hat{m}_{0+} \simeq |\langle\psi|0+\rangle|^2 = \frac{a+b}{2} \tag{31}$$

$$\hat{m}_{0-} \simeq |\langle\psi|0-\rangle|^2 = \frac{a-b}{2} \tag{32}$$

$$\hat{m}_{1+} \simeq |\langle\psi|1+\rangle|^2 = \frac{c+d}{2} \tag{33}$$

$$\hat{m}_{1-} \simeq |\langle\psi|1-\rangle|^2 = \frac{c-d}{2} \tag{34}$$

Finally, we apply a Hadamard gate to the first qubit with $U^2_{\text{add}} = H \otimes I$ and measure:

$$\hat{m}_{+0} \simeq |\langle\psi|+0\rangle|^2 = \frac{a+c}{2} \tag{35}$$

$$\hat{m}_{+1} \simeq |\langle\psi|+1\rangle|^2 = \frac{a-c}{2} \tag{36}$$

$$\hat{m}_{-0} \simeq |\langle\psi|-0\rangle|^2 = \frac{b+d}{2} \tag{37}$$

$$\hat{m}_{-1} \simeq |\langle\psi|-1\rangle|^2 = \frac{b-d}{2} \tag{38}$$

To determine the signs:

1. $a$ is taken as positive.
2. If $\hat{m}_{0+} > \hat{m}_{0-}$, then $b$ is also positive; otherwise, $b$ is negative.
3. If $\hat{m}_{+0} > \hat{m}_{-0}$, then $c$ is also positive; otherwise, $c$ is negative.
4. Finally, the sign of $d$ is is determined by the signs of $b$ and $c$. If $\hat{m}_{1+} > \hat{m}_{1-}$(or $\hat{m}_{-0} > \hat{m}_{-1}$), then $d$ has the same sign as $b$(or $c$); otherwise, it has the opposite sign.

$a$ is fixed to positive, and if $\hat{m}_{0+} > \hat{m}_{0-}$, then $b$ is also positive, otherwise $b$ is negative.

After determining, $b$ and $c$, we can determine the sign of $d$.

**b) Now, explain how you would adapt this measurement protocol if the amplitudes $a, b, c, d$ are complex numbers. Note, that we can only reconstruct the wavefunction up to a global phase, so that we can always consider $a$ to be a positive real number.**

The two-qubit pure state can be expressed as:

$$|\psi\rangle = a\,|00\rangle + be^{i\theta_b}\,|01\rangle + ce^{i\theta_c}\,|10\rangle + de^{i\theta_b}\,|11\rangle \tag{39}$$

where $a, b, c$ and $d$ are positive real numbers, and $\theta_b, \theta_c, \theta_d$ are phases in the range $0 \leq \theta_b, \theta_c, \theta_d < 2\pi$.

**Initial Measurement** Begin with the circuit without any additional operations, $U^0_{\text{add}} = I_4$. This measures in the standard basis $\{|00\rangle\langle 00|, |01\rangle\langle 01|, |10\rangle\langle 10|, |11\rangle\langle 11|\}$, yielding $a^2, b^2, c^2$ and $d^2$:

$$\hat{m}_{00} \simeq |\langle\psi|00\rangle|^2 = a^2 \tag{40}$$

$$\hat{m}_{01} \simeq |\langle\psi|01\rangle|^2 = b^2 \tag{41}$$

$$\hat{m}_{10} \simeq |\langle\psi|10\rangle|^2 = c^2 \tag{42}$$

$$\hat{m}_{11} \simeq |\langle\psi|11\rangle|^2 = d^2 \tag{43}$$

**Hadamard on First Qubit** Next, apply a Hadamard gate on the first qubit, $U^1_{\text{add}} = I_2 \otimes H$:

$$\hat{m}_{0+} \simeq |\langle\psi|0+\rangle|^2 = \left|\frac{a + be^{i\theta_b}}{\sqrt{2}}\right|^2 \tag{44}$$

$$\hat{m}_{0-} \simeq |\langle\psi|0-\rangle|^2 = \left|\frac{a - be^{i\theta_b}}{\sqrt{2}}\right|^2 \tag{45}$$

$$\hat{m}_{1+} \simeq |\langle\psi|1+\rangle|^2 = \left|\frac{ce^{i\theta_c} + de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{46}$$

$$\hat{m}_{1-} \simeq |\langle\psi|1-\rangle|^2 = \left|\frac{ce^{i\theta_c} - de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{47}$$

From these measurements, we can determine::

$$\cos\theta_b = \frac{\hat{m}_{0+} - \hat{m}_{0-}}{2ab} \tag{48}$$

$$\cos(\theta_c - \theta_d) = \frac{\hat{m}_{1+} - \hat{m}_{1-}}{2cd} \tag{49}$$

$$\tag{50}$$

**Hadamard and S gate on First Qubit** Apply both a Hadamard and S gate on the first qubit,, $U^2_{\text{add}} = I_2 \otimes SH$:

$$\hat{m}_{0i} \simeq |\langle\psi|0i\rangle|^2 = \left|\frac{a + ce^{i\theta_c + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{51}$$

$$\hat{m}_{0-i} \simeq |\langle\psi|0-i\rangle|^2 = \left|\frac{be^{i\theta_b} + de^{i\theta_d + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{52}$$

$$\hat{m}_{1i} \simeq |\langle\psi|1i\rangle|^2 = \left|\frac{a - ce^{i\theta_c - \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{53}$$

$$\hat{m}_{1-i} \simeq |\langle\psi|1-i\rangle|^2 = \left|\frac{be^{i\theta_b} - de^{i\theta_d - \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{54}$$

These measurements yield:

$$\sin\theta_b = \frac{-\hat{m}_{0i} + \hat{m}_{0-i}}{2ab} \tag{55}$$

$$\sin(\theta_c - \theta_d) = \frac{\hat{m}_{1i} - \hat{m}_{1-i}}{2cd} \tag{56}$$

$$\tag{57}$$

**Fourth and Fifth Measurements** Repeat similar operations on the second qubit to measure $\theta_c$ and $\theta_d$ using $U^3_{\text{add}} = H \otimes I_2$ and $U^4_{\text{add}} = SH \otimes I_2$:

$$\hat{m}_{+0} \simeq |\langle\psi|{+}0\rangle|^2 = \left|\frac{a + ce^{i\theta_c}}{\sqrt{2}}\right|^2 \tag{58}$$

$$\hat{m}_{+1} \simeq |\langle\psi|{+}1\rangle|^2 = \left|\frac{be^{i\theta_b} + de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{59}$$

$$\hat{m}_{-0} \simeq |\langle\psi|{-}0\rangle|^2 = \left|\frac{a - ce^{i\theta_c}}{\sqrt{2}}\right|^2 \tag{60}$$

$$\hat{m}_{-1} \simeq |\langle\psi|{-}1\rangle|^2 = \left|\frac{be^{i\theta_b} - de^{i\theta_d}}{\sqrt{2}}\right|^2, \tag{61}$$

and

$$\hat{m}_{i0} \simeq |\langle\psi|i0\rangle|^2 = \left|\frac{a + ce^{i\theta_c + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{62}$$

$$\hat{m}_{i1} \simeq |\langle\psi|i1\rangle|^2 = \left|\frac{be^{i\theta_b} + de^{i\theta_d + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{63}$$

$$\hat{m}_{-i0} \simeq |\langle\psi|{-}i0\rangle|^2 = \left|\frac{a - ce^{i\theta_c + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{64}$$

$$\hat{m}_{-i1} \simeq |\langle\psi|{-}i1\rangle|^2 = \left|\frac{be^{i\theta_b} - de^{i\theta_d + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{65}$$

From these measurements, we obtain:

$$\cos\theta_c = \frac{\hat{m}_{+0} - \hat{m}_{-0}}{2ac} \tag{66}$$

$$\cos(\theta_b - \theta_d) = \frac{\hat{m}_{+1} - \hat{m}_{-1}}{2bd} \tag{67}$$

$$\tag{68}$$

and

$$\sin\theta_c = \frac{-\hat{m}_{i0} + \hat{m}_{-i0}}{2ac} \tag{69}$$

$$\sin(\theta_b - \theta_d) = \frac{\hat{m}_{i1} - \hat{m}_{-i1}}{2bd} \tag{70}$$

$$\tag{71}$$

**Error Handling** Issues arise if any of $a, b, c$, or $d$ is zero, as this makes certain equations inapplicable. Specifically:

- if $a = 0$, Eq. 48, Eq. 55, Eq. 66, Eq. 69 are not available.
- if $b = 0$, Eq. 48, Eq. 55, Eq. 67, Eq. 70 are not available.
- if $c = 0$, Eq. 49, Eq. 56, Eq. 66, Eq. 69 are not available.
- if $d = 0$, Eq. 49, Eq. 56, Eq. 67, Eq. 70 are not available.

When $x \in b, c, d$ is zero, setting $\theta_x = 0$ allows us to determine the remaining phase parameters using available equations. When $a = 0$, we set $\theta_b = 0$ and resolve the other phases similarly. However, if both $a = d = 0$ or $b = c = 0$, phase differences cannot be determined using the above method.

To address these cases, after running the initial circuit to check for zeroes, we add a CNOT gate with $q_0$ as the control qubit and $q_1$ as the target qubit to the subsequent circuits. This modifies the circuits as follows:

$$U^1_{\text{add}} = (I_2 \otimes H)CNOT_{0,1} \tag{72}$$

$$U^2_{\text{add}} = (I_2 \otimes SH)CNOT_{0,1} \tag{73}$$

$$U^3_{\text{add}} = (H \otimes I_2)CNOT_{0,1} \tag{74}$$

$$U^4_{\text{add}} = (SH \otimes I_2)CNOT_{0,1} \tag{75}$$

With this modification, the coefficients of $|01\rangle$ and $|11\rangle$ are swapped, allowing us to treat cases with $a = d = 0$ and $b = c = 0$ as if they were $a = b = 0$ and $c = d = 0$. Once the state vector is reconstructed, we simply revert the coefficients of $|01\rangle$ and $|11\rangle$.

**c) Now write a program in Qiskit which will generate this unknown quantum state. Use the same format as for part I.** To generate a random quantum state, the following code is used:

```
def create_2q_target_circuit(qc_data):
    qc = qiskit.QuantumCircuit(2)
    for i in range(2):
        qc.ry(qc_data[2*i+1],i)
```

```
        qc.rz(qc_data[2*i+1],i)
    qc.cz(0,1)
    qc.rz(qc_data[4],1)
    qc.cx(0,1)
    for i in range(2):
        qc.ry(qc_data[2*i+5],i)
        qc.rz(qc_data[2*i+5],i)
    return qc
```

As described in part (b), we can reconstruct the state vector with five measurements. I implemented the following code to reconstruct the state vector.

```
def TwoQubitQST(U_hidden, num_shots:int=100):
  hidden_circuit = U_hidden.copy()

  qc1 = qiskit.QuantumCircuit(2)
  circ1 = hidden_circuit.compose(qc1)
  circ1.measure_all()
  #print(circ1.draw())
  result1 = MeasurementResultHandler(backend
              .run(circ1, shots=num_shots)
              .result().data()["counts"], 2, num_shots)
  m_00 = result1.get("0x0"); m_01 = result1.get("0x1");
    m_10 = result1.get("0x2"); m_11 = result1.get("0x3")
  if m_00 == 1:
    return np.array([1,0,0,0], dtype=np.complex64)
  if m_01 == 1:
    return np.array([0,1,0,0], dtype=np.complex64)
  if m_10 == 1:
    return np.array([0,0,1,0], dtype=np.complex64)
  if m_11 == 1:
    return np.array([0,0,0,1], dtype=np.complex64)

  qc2 = qiskit.QuantumCircuit(2)
  if (m_00 == 0 and m_11 == 0) or (m_01 == 0 and m_10 == 0):
    qc2.cx(0,1) # swap b, d

  qc2.h(0)
  circ2 = hidden_circuit.compose(qc2)
  circ2.measure_all()
  #print(circ2.draw())
```

```
qc3 = qiskit.QuantumCircuit(2)
if (m_00 == 0 and m_11 == 0) or (m_01 == 0 and m_10 == 0):
  qc3.cx(0,1) # swap b, d
qc3.s(0)
qc3.h(0)
circ3 = hidden_circuit.compose(qc3)
circ3.measure_all()
#print(circ3.draw())

qc4 = qiskit.QuantumCircuit(2)
if (m_00 == 0 and m_11 == 0) or (m_01 == 0 and m_10 == 0):
  qc4.cx(0,1) # swap b, d
qc4.h(1)
circ4 = hidden_circuit.compose(qc4)
circ4.measure_all()
#print(circ4.draw())

qc5 = qiskit.QuantumCircuit(2)
if (m_00 == 0 and m_11 == 0) or (m_01 == 0 and m_10 == 0):
  qc5.cx(0,1) # swap b, d
qc5.s(1)
qc5.h(1)
circ5 = hidden_circuit.compose(qc5)
circ5.measure_all()
#print(circ5.draw())

result2 = MeasurementResultHandler(backend
            .run(circ2, shots=num_shots).result()
            .data()["counts"], 2, num_shots)
result3 = MeasurementResultHandler(backend
            .run(circ3, shots=num_shots).result()
            .data()["counts"], 2, num_shots)
result4 = MeasurementResultHandler(backend
            .run(circ4, shots=num_shots).result()
            .data()["counts"], 2, num_shots)
result5 = MeasurementResultHandler(backend
            .run(circ5, shots=num_shots).result()
            .data()["counts"], 2, num_shots)

a = np.sqrt(m_00); b = np.sqrt(m_01);
c = np.sqrt(m_10); d = np.sqrt(m_11)
if (m_00 == 0 and m_11 == 0) or (m_01 == 0 and m_10 == 0):
  d = np.sqrt(m_01); b = np.sqrt(m_11)
```

```
statevector = np.array([a,b,c,d], dtype=np.complex64)
theta_01 = 0; theta_10 = 0; theta_11 = 0

executing_Eq = {i for i in range(1,9)}
related_Eq = {
  "a" : {1,3,5,7},
  "b" : {1,3,6,8},
  "c" : {2,4,5,7},
  "d" : {2,4,6,8}
}
if a == 0:
  executing_Eq-= related_Eq["a"]
if b == 0:
  executing_Eq-= related_Eq["b"]
if c == 0:
  executing_Eq-= related_Eq["c"]
if d == 0:
  executing_Eq-= related_Eq["d"]

m_0p = result2.get("0x0"); m_0m = result2.get("0x1");
  m_1p = result2.get("0x2"); m_1m = result2.get("0x3")
m_0P = result3.get("0x0"); m_0M = result3.get("0x1");
  m_1P = result3.get("0x2"); m_1M = result3.get("0x3")
m_p0 = result4.get("0x0"); m_p1 = result4.get("0x1");
  m_m0 = result4.get("0x2"); m_m1 = result4.get("0x3")
m_P0 = result5.get("0x0"); m_P1 = result5.get("0x1");
  m_M0 = result5.get("0x2"); m_M1 = result5.get("0x3")

if 1 in executing_Eq:
  # eq 1.
  cos_00_01 = (m_0p - m_0m) / (2*a*b)
if 2 in executing_Eq:
  # eq 2.
  cos_10_11 = (m_1p - m_1m) / (2*c*d)

if 3 in executing_Eq:
  # eq 3.
  sin_00_01 = (-m_0P + m_0M) / (2*a*b)
if 4 in executing_Eq:
  # eq 4.
  sin_10_11 = (m_1P - m_1M) / (2*c*d)
```

```
if 5 in executing_Eq:
  # eq 5.
  cos_00_10 = (m_p0 - m_m0) / (2*a*c)
if 6 in executing_Eq:
  # eq 6.
  cos_01_11 = (m_p1 - m_m1) / (2*b*d)

if 7 in executing_Eq:
  # eq 7.
  sin_00_10 = (-m_P0 + m_M0) / (2*a*c)
if 8 in executing_Eq:
  # eq 8.
  sin_01_11 = (m_P1 - m_M1) / (2*b*d)


if a != 0:
  if b != 0:
    theta_01 = np.arctan2(sin_00_01, cos_00_01)
    theta_01 = theta_01 if theta_01 >= 0
                 else theta_01 + 2*np.pi
    if c != 0:
      theta_10 = np.arctan2(sin_00_10, cos_00_10)
      theta_10 = theta_10 if theta_10 >= 0
                   else theta_10 + 2*np.pi
      if d != 0:
        if b < c:

          theta_10_11= np.arctan2(sin_10_11, cos_10_11)
          theta_10_11 = theta_10_11 if theta_10_11 >= 0
                          else theta_10_11 + 2*np.pi

          theta_11 = theta_10 - theta_10_11
          theta_11 = theta_11 if theta_11 >= 0
                       else theta_11 + 2*np.pi
        else:
          theta_01_11= np.arctan2(sin_01_11, cos_01_11)
          theta_01_11 = theta_01_11 if theta_01_11 >= 0
                          else theta_01_11 + 2*np.pi

          theta_11 = theta_01 - theta_01_11
          theta_11 = theta_11 if theta_11 >= 0
                       else theta_11 + 2*np.pi
```

```
        #else:
      else:
        if d!=0:
          theta_01_11= np.arctan2(sin_01_11, cos_01_11)
          theta_01_11 = theta_01_11 if theta_01_11 >= 0
                          else theta_01_11 + 2*np.pi

          theta_11 = theta_01 - theta_01_11
          theta_11 = theta_11 if theta_11 >= 0
                        else theta_11 + 2*np.pi
        #else:
    else:
      if c != 0:
        theta_10 = np.arctan2(sin_00_10, cos_00_10)
        theta_10 = theta_10 if theta_10 >= 0
                    else theta_10 + 2*np.pi

        if d != 0:
          theta_10_11= np.arctan2(sin_10_11, cos_10_11)
          theta_10_11 = theta_10_11 if theta_10_11 >= 0
                          else theta_10_11 + 2*np.pi

          theta_11 = theta_10 - theta_10_11
          theta_11 = theta_11 if theta_11 >= 0
                        else theta_11 + 2*np.pi
      #else:

  else:
    if b != 0:
      if c != 0 and d != 0:
        theta_01_11= np.arctan2(sin_01_11, cos_01_11)
        theta_01_11 = theta_01_11 if theta_01_11 >= 0
                        else theta_01_11 + 2*np.pi

        theta_11 = - theta_01_11
        theta_11 = theta_11 if theta_11 >= 0
                    else theta_11 + 2*np.pi

        theta_10_11= np.arctan2(sin_10_11, cos_10_11)
        theta_10_11 = theta_10_11 if theta_10_11 >= 0
                        else theta_10_11 + 2*np.pi

        theta_10 = theta_11 + theta_10_11
```

```
        theta_10 = theta_10 if theta_10 >= 0
                    else theta_10 + 2*np.pi

    else:
      theta_10_11= np.arctan2(sin_10_11, cos_10_11)
      theta_10_11 = theta_10_11 if theta_10_11 >= 0
                     else theta_10_11 + 2*np.pi

      theta_11 = - theta_10_11
      theta_11 = theta_11 if theta_11 >= 0
                   else theta_11 + 2*np.pi

  statevector[1] *= np.exp(1j*theta_01)
  statevector[2] *= np.exp(1j*theta_10)
  statevector[3] *= np.exp(1j*theta_11)

  if (m_00 == 0 and m_11 == 0) or (m_01 == 0 and m_10 == 0):
    statevector[1], statevector[3] = \
        statevector[3], statevector[1]

  return statevector
```

Next, we need to determine the circuit parameters and ansatz. It's known that one CNOT gate suffices to prepare a two-qubit state, so I used the following circuit ansatz.

For a bipartite pure state $|\psi\rangle$, there exists an orthonormal basis $|i\rangle_1$ for the first party and $|i\rangle_2$ for the other party such that

$$|\psi\rangle = \sum_i \lambda_i |i\rangle_1 \otimes |i\rangle_2 \tag{76}$$

where $\lambda_i$ are non-negative real numbers with $\sum_i \lambda_i^2 = 1$. This is known as the Schmidt decomposition. Through singular value decomposition, $\lambda = UDV$, where $D$ is a diagonal matrix of non-negative elements and $U$ and $V$ are unitary matrices. Then,

$$|\psi\rangle = \sum_{i,j,k} U_{ji} D_{ii} V_{ik} |j\rangle |k\rangle . \tag{77}$$

By defining $|i_1\rangle = \sum_j U_{ji} |j\rangle$, $|i_2\rangle = \sum_j V_{ik} |k\rangle$, and $\lambda_i = d_{ii}$, we can express the state in the form of Eq. 76. For convenience, we make $\langle 0|i_1\rangle$ and $\langle 0|i_2\rangle$ real by factoring out the global phase; this also allows $\lambda_i$ to be non-real if needed. Additionally, setting $\lambda_i' = \lambda_i \frac{|\lambda_0|}{\lambda_0}$ makes $\lambda_0'$ real.

Here is the strategy to design the circuit ansatz and set the circuit parameters: Starting from the initial state $|\psi\rangle = |00\rangle$, apply a local unitary

gate on the first qubit to get:

$$|\psi\rangle \to \lambda_0' |00\rangle + \lambda_1' |01\rangle \tag{78}$$

Then, apply a CNOT gate:

$$|\psi\rangle \to \lambda_0' |00\rangle + \lambda_1' |11\rangle \tag{79}$$

Applying local unitary operations $U_0$ and $U_1$ on each qubit results in:

$$U_1 |0\rangle_1 \to |i^0\rangle_1 \qquad U_1 |1\rangle_1 \to |i^1\rangle_1 \tag{80}$$

$$U_2 |0\rangle_2 \to |j^0\rangle_2 \qquad U_2 |1\rangle_2 \to |j^1\rangle_2 \tag{81}$$

where upper index is inserted to distinguish the basis. The state then takes the form of the Schmidt decomposition in Eq. 76.

Next, we address the required local unitaries. To achieve a unitary transformation $|0_1\rangle \to \lambda_0' |0\rangle + \lambda_1' |1\rangle$. we use an $RY$-$RZ$ decomposition, as done in part I:

$$R_z(\beta_1) R_y(\alpha_1) |0\rangle = \cos\frac{\alpha}{2} |0\rangle + e^{i\beta} \sin\frac{\alpha}{2} \tag{82}$$

Then,

$$\alpha_1 = 2 \arccos \lambda_0' = 2 \arcsin |\lambda_1'| \tag{83}$$

$$\beta_1 = -i \ln \frac{\lambda_1}{|\lambda_1|}. \tag{84}$$

The local unitaries $U_1$ and $U_2$ are similarly implemented, effectively preparing the state by $RY$ and $RZ$ gates:

$$\alpha_2 = 2 \arccos \langle 0|i_1\rangle \tag{85}$$

$$\beta_2 = -i \ln \frac{\langle 1|i_1\rangle}{|\langle 1|i_1\rangle|} \tag{86}$$

$$\alpha_3 = 2 \arccos \langle 0|i_2\rangle \tag{87}$$

$$\beta_3 = -i \ln \frac{\langle 1|i_2\rangle}{|\langle 1|i_2\rangle|} \tag{88}$$

Therefore, my algorithm procedure is as following:

**Phase 1.** Perform the Schmidt decomposition on the reconstructed vector using the Schmidt decomposition method available in Numpy. The columns of $U$ form the basis $|i_1\rangle$, and the rows of $V^\dagger$ form the basis $|i_2\rangle$.

**Phase 2.** Adjust the global phase for all Schmidt-decomposed basis vectors so that the first element of each basis vector is real. This is achieved by dividing the global phase from the state vector and applying it to $\lambda_i$. Then, perform an additional phase adjustment to ensure that $\lambda_0'$ is real.

**Phase 3.** In this phase, we use the values obtained from the Schmidt decomposition to determine the circuit parameters for the circuit, by using Eq. 83 and Eq. 88.

**Phase 4.** Once the parameters are calculated, we construct the circuit to prepare the target state. Starting from the initial state $|00\rangle$, we:

1. Apply the Ry and Rz gates with the calculated parameters to the first qubit, which sets the appropriate amplitudes for the first qubit.
2. Insert a CNOT gate to entangle the qubits, which maps the state to $\lambda'0\,|00\rangle + \lambda'1\,|11\rangle$, capturing the entangled component of the target state.
3. Apply additional Ry and Rz gates on each qubit, according to the basis states $|i\rangle_1$ and $|i\rangle_2$ obtained from the Schmidt decomposition.

**Exception** In cases where there are not exactly two non-zero Schmidt coefficients, the state is separable and does not require a two-qubit gate. When the Schmidt coefficient is 1, only local unitary operations on each qubit are needed, as outlined in in Eq. 80 and Eq. 81. Parameter extraction follows the same steps as in Eq. 85 to Eq. 88.

Here, I implemented the following code to construct a quantum circuit from the given state vector.

```
def TwoQubitCircuit(statevector, eps=1e-4):

  # Schmidt decomposition
  a_jk = statevector.reshape(2,2)
  u, d, vh = np.linalg.svd(a_jk)
  d = d.astype(np.complex64)


  if d[0] == 0 or d[1] == 0:
    idx = 0 if d[1] == 0 else 1

    e2_1 = u[:,idx]
    e1_1 = vh[idx,:]

    # Ensure e2_1[0], ... be real, positive number
    phase1 = e2_1[0] / np.linalg.norm(e2_1[0]) if np.linalg.no
```

```
    theta_2 = 2*np.arccos(e2_1[0].real)
    phi_2 = (-1j * np.log(e2_1[1]/np.sin(theta_2/2))).real if np.abs(np.sin(theta_2

    U_gen = qiskit.QuantumCircuit(2)
    U_gen.ry(theta_1,0)
    U_gen.rz(phi_1,0)
    U_gen.ry(theta_2,1)
    U_gen.rz(phi_2,1)

    return U_gen

  else:
    e2_1 = u[:,0]; e2_2 = u[:,1]
    e1_1 = vh[0,:]; e1_2 = vh[1,:]
    # Ensure e2_1[0], ... be real, positive number
    phase1 = e2_1[0] / np.linalg.norm(e2_1[0]) if np.linalg.norm(e2_1[0]) != 0 else
    d[0] *= phase1 * phase2
    e2_1 /= phase1; e1_1 /= phase2
    phase1 = e2_2[0] / np.linalg.norm(e2_2[0]) if np.linalg.norm(e2_2[0]) != 0 else
    d[1] *= phase1 * phase2
    e2_2 /= phase1; e1_2 /= phase2

    phase_d = d[0] / np.linalg.norm(d[0])
    d /= phase_d

    theta = 2 * np.arctan2(d[0].imag, d[0].real)
    phi = (-1j*np.log(d[1]/np.sin(theta/2))).real if np.abs(np.sin(theta/2)) != 0 e

    theta_1 = 2*np.arccos(e1_1[0].real)
    phi_1 = (-1j * np.log(e1_1[1]/np.sin(theta_1/2))).real if np.abs(np.sin(theta_1

    theta_2 = 2*np.arccos(e2_1[0].real)
    phi_2 = (-1j * np.log(e2_1[1]/np.sin(theta_2/2))).real if np.abs(np.sin(theta_2

    U_gen = qiskit.QuantumCircuit(2)
    U_gen.ry(theta,0)
    U_gen.rz(phi,0)
    U_gen.cx(0,1)
    U_gen.ry(theta_1,0)
    U_gen.rz(phi_1,0)
    U_gen.ry(theta_2,1)
    U_gen.rz(phi_2,1)
```

```
    return U_gen
```

## Part III

**Describe how you could extend the protocol developed in part's I) and II) to a general n qubit quantum state. How many different measurements would you need to apply?**

Reconstructing a quantum state involves two main parts: state reconstruction and circuit construction. An arbitrary $n$-qubit pure state can be represented as:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} a_i e^{\theta_i} |i\rangle \tag{89}$$

where $i$ is a bit string, $a_i$ are real positive numbers constrained by $\sum_i |a_i|^2 = 1$, $0 \leq \theta_i < 2\pi$, and $\theta_0 = 0$ for simplicity.

First, state reconstruction consists of two steps: extracting amplitudes and measuring phases. The amplitudes $a_i$ can be obtained by measuring in the basis $|0\rangle, |1\rangle, \ldots, |2^n - 1\rangle$, requiring no additional operations before or after measurement. The phase difference between $|i\rangle$ and $|j\rangle$, where the Hamming distance between $i$ and $j$ is 1, can be determined by applying unitary operations on the $i$-th qubit, using $U = H_i$ and $U = S_i H_i$. This enables calculation of $\theta_i - \theta_j$ as in Parts I and II. To determine the complete phase information, this measurement process with $U = H_i$ and $U = S_i H_i$ should be repeated for all qubits, resulting in $2n + 1$ circuits required for quantum state tomography (QST) on an $n$-qubit pure state.

Once the state vector is reconstructed, we can determine the circuit ansatz and parameters. For up to three qubits, standard ansatz methods exist for arbitrary state preparation. The one- and two-qubit cases are covered in Parts I and II. For three qubits, it is well-known that three CNOT gates are sufficient for preparing an arbitrary three-qubit pure state[4]. For $n \geq 4$, additional strategies are described in the following references[2, 5].

Consider $n = 2k$ case:

**Phase 1** Use Schmidt decomposition to express the state as:

$$|\psi\rangle = \sum_{i=0}^{2^k-1} \alpha_i |i\rangle_1 |i\rangle_2 \tag{90}$$

**Phase 2** Apply local unitary gates to the initial state $|\psi\rangle = |0\rangle^{\otimes n}$, so that:

$$ket\psi \to \left( \sum_{i=0}^{2^k-1} \alpha_i |i\rangle \right) \otimes |0\rangle^{\otimes k} . \tag{91}$$

**Phase 3** Perform $k$ CNOT gates, with qubit $i$ (for $i = 0$ to $i = k-1$) as the control qubit and qubit $i+k$ as the target qubit, to achieve:

$$|\psi\rangle \to \sum_{i=0}^{2^k-1} \alpha_i |i\rangle_1 |i\rangle_2 \tag{92}$$

**Phase 4** Apply $k$-qubit unitary operations $U_1$ and $U_2$ on each subsystem, such that:

$$U_1 |i\rangle_1 = |i\rangle_1 \qquad\qquad U_2 |i\rangle_2 = |i\rangle_2 \tag{93}$$

In the case of $n = 2k-1$, we divide the system into bipartite state by $k-1$ qubits and $k$ qubits.

**Phase 1** Use Schmidt decomposition to express the state as:

$$|\psi\rangle = \sum_{i=0}^{2^{k-1}-1} \alpha_i |i\rangle_1 |i\rangle_2 . \tag{94}$$

**Phase 2** Apply local unitary gates to the initial state $|\psi\rangle = |0\rangle^{\otimes n}$, so that:

$$ket\psi \to \left( \sum_{i=0}^{2^{k-1}-1} \alpha_i |i\rangle \right) \otimes |0\rangle^{\otimes k} . \tag{95}$$

**Phase 3** Perform $k$ CNOT gates, with qubit $i$ (for $i = 0$ to $i = k-2$) as the control and qubit $i+k$ as the target, yielding:

$$|\psi\rangle \to \sum_{i=0}^{2^{k-1}-1} \alpha_i |i\rangle |i\rangle \tag{96}$$

At this point, the final qubit remains in the $|0\rangle$ state, marking a key difference from the even case.

**Phase 4** Apply $k-1$- and $k$-qubit unitary operations $U_1$ and $U_2$ on each subsystem respectively, such that:

$$U_1 |i\rangle_1 = |i\rangle_1 \qquad\qquad U_2 |i\rangle_2 = |i\rangle_2 \tag{97}$$

Phases 1 and 2 correspond to state preparation, which can be performed recursively. The challenge lies in constructing the $k$-qubit unitary operation. This procedure is well described in the follwing reference[2]. To briefly explain, the $k$-qubit unitary operation $U \in \mathbb{C}^{2^k \times 2^k}$ can be represented as:

$$U = \begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} B_0 & 0 \\ 0 & B_1 \end{pmatrix} \tag{98}$$

where $A_0, A_1, B_0, B_1 \in \mathbb{C}^{2^{k-1} \times 2^{k-1}}$ are unitary matrices and $C, S$ are real diagonal matrics such that $C^2 + S^2 = 1$. This decomposition is known as Cosine-Sine Decomposition(CSD). To perform the CSD, partition $U$ into four blocks:

$$U = \begin{pmatrix} U_{00} & U_{01} \\ U_{10} & U_{11} \end{pmatrix} \tag{99}$$

Then, unitary matrices $A_0, A_1, B_0, B_1$ satisfy:

$$U = \begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} B_0 & 0 \\ 0 & B_1 \end{pmatrix}. \tag{100}$$

This step is essentially performing four singular value decompositions:

$$U_{00} = A_0 C B_0^\dagger \tag{101}$$
$$U_{01} = A_0 S B_1^\dagger \tag{102}$$
$$U_{10} = A_1 (-S) B_0^\dagger \tag{103}$$
$$U_{11} = A_1 C B_1^\dagger \tag{104}$$

Next, it becomes a controlled-$U$ gate followed by a controlled-$R_y$ gate:

$$\begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} = |0\rangle\langle 0| \otimes A_0 + |0\rangle\langle 0| \otimes A_1 \tag{105}$$
$$\begin{pmatrix} B_0 & 0 \\ 0 & B_1 \end{pmatrix} = |0\rangle\langle 0| \otimes B_0 + |0\rangle\langle 0| \otimes B_1 \tag{106}$$
$$\begin{pmatrix} C & -S \\ S & C \end{pmatrix} = \sum I \otimes |0\rangle\langle 0| + R_y \otimes |1\rangle\langle 1| \tag{107}$$

The controlled unitary operation, $\begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix}$ can be expressed as:

$$\begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} = \begin{pmatrix} V & 0 \\ 0 & V \end{pmatrix} \begin{pmatrix} D & 0 \\ 0 & D^\dagger \end{pmatrix} \begin{pmatrix} W & 0 \\ 0 & W \end{pmatrix} \tag{108}$$

where $V, W \in \mathbb{C}^{2^{k-2} \times 2^{k-2}}$ are unitary matrices, and $D$ is a diagonal matrix. $V$, $W$, and $D$ can be found through eigen-decomposition:

$$A_0 A_1^\dagger = V D^2 V^\dagger \tag{109}$$

$$W = D V^\dagger A_1 \tag{110}$$

Thus, the controlled unitary can be written as:

$$\begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} = (I \otimes V)(|0\rangle\langle 0| \otimes D + |1\rangle\langle 1| \otimes D^\dagger)(I \otimes W) \tag{111}$$

The unitary operations $V$ and $W$ are in $\mathbb{C}^{2^{k-2} \times 2^{k-2}}$, indicating that they can be iteratively decomposed using CSD. Meanwhile, $(|0\rangle\langle 0| \otimes D + |1\rangle\langle 1| \otimes D^\dagger)$ represents a controlled-$Z$ gate.

## Problem 2 : Graph States

In some cases, one can reconstruct a quantum state using only a limited number of measurements, regardless of how many qubits are in the quantum system. One of these cases is when the quantum states are generated using only "Clifford gates" (these are essentially quantum circuits that use only H,S, CNOT and CZ gates). In this problem, we will perform quantum state tomography on a subset of Clifford states known as "graph states". A graph state on n qubits is any state which can be generated using a circuit of the form

$$|\psi\rangle = \left( \prod_{(i,j) \in G} CZ_{i,j} \right) H^{\otimes n} |0\rangle \tag{112}$$

Where $CZ_{ij}$ is a controlled-Z gate applied on qubits $i$ and $j$, and $(i,j)$ are edges of a hidden graph $G$. Different graphs, $G$, create different graph states. Our goal is to determine generate the graph state, $|\psi\rangle$, without knowing the specific graph which was used.

### a) Look into the properties of graph states. Graph states are stabilizer states with a specific structure. Describe the key properties of stabilizer states. Using the stabilizer properties of graph states, describe how one can perform tomography on graph states.

Graph states are a class of stabilizer states with a defined structure. In an $n$-qubit system, stabilizer states are characterized by $n$ stabilizers, each being an $N$-fold tensor product of Pauli operators (up to a factor of $\pm 1, \pm i$). A stabilizer is an operator that leaves the state invariant, meaning the state is a simultaneous $+1$ eigenstate of all stabilizers:

$$S_i |\psi\rangle = |\psi\rangle \tag{113}$$

For graph states, the stabilizer associated with each vertex $i$ is given by:

$$S_i = X_i \prod_{(i,j)\in G} Z_j \tag{114}$$

where each stabilizer $S_i$ corresponds to qubit $i$ and its neighboring qubits in the graph $G$.

It is important to note that a graph state is a simultaneous $+1$ eigenstate of its stabilizers, meaning it can be written as a linear combination of the stabilizer eigenbasis. The stabilizer eigenbasis takes the form:

$$|\pm\rangle_i \bigotimes_{j\neq i} |b_j\rangle \tag{115}$$

where the condition $b_i \bigoplus_{j\in(i,j)} b_j = 0$, holds, imposed by the $+1$ eigenvalue requirement (with $+$ and $-$ corresponding to 0 and 1, respectively).

To perform measurements in this basis for a given vertex $i$, apply a Hadamard gate $H$ to qubit $i$ before measurement. After collecting multiple measurement outcomes, identify the set of $j$ such that $b_i \oplus \bigoplus_{j\in(i,j)} b_j = 0$. By repeating this process for each qubit, we can reconstruct the graph of the $n$-qubit system by running the circuit $n$ times.

**b) You may use the following code snippet to generate a random graph state You may set `num_qubits` $< 20$. Following the format of Problem 1, measure $U_{\text{hidden}}$ in $\leq$ 20 bases by appendix quantum circuits to $U_{\text{hidden}}$ and taking no more than 100 shots per basis. Then create a new circuit $U_{\text{gen}}$ such that $U_{\text{gen}}^{\dagger} U_{\text{hidden}} |0\rangle = |0\rangle$.**

```
def generate_clifford_circ(num_qubits, depth):
    U_hidden = qiskit.QuantumCircuit(num_qubits)
    for i in range(num_qubits):
        U_hidden.h(i)
    count = 0
    s1s2 = {-1,-1}
    while(count < depth):
        site_i = int(np.random.rand()*num_qubits)
        site_j = int(np.random.rand()*num_qubits)
        if(site_i == site_j or {site_i, site_j} == s1s2):
            continue
        print(site_i, site_j, count)
        count += 1
        s1s2 = {site_i, site_j}
```

```
        U_hidden.cz(site_i, site_j)
    return U_hidden
```

As outlined in part a), I implemented the following algorithm:

**Phase 1** Apply a Hadamard gate to the $i$-th site after generating the graph state and before measurement.

**Phase 2** Extract the resulting bit strings from the measurements.

**Phase 3** Identify the set of $j$ such that $b_i \bigoplus_{j\in(i,j)} \oplus b_j = 0$. Checking all possible sets of $(i,j)$ is feasible since the total number of combinations to check is $n2^{n-1}$, which amounts to at most $20 \times 2^19 \simeq 10485760$ per graph state.

Here are the implemented code:

```
import itertools

def GraphStateQST(U_hidden, num_qubits, num_shots=100):
  hidden_circuit = U_hidden.copy()

  graph = [] # G: set of (i,j)
  for i in range(num_qubits):
    # Phase 1
    qc = qiskit.QuantumCircuit(num_qubits)
    qc.h(i)
    circ = hidden_circuit.compose(qc)
    circ.measure_all()

    # Phase 2
    # Extract bit string
    result = backend.run(circ, shots=num_shots)
            .result().data()["counts"]
    result = [int(i,16) for i in result.keys()]

    # Phase 3
    # Generating possible set of (i,j)
    qubit_index = [idx for idx in range(num_qubits)]
    qubit_index.pop(i)

    for k in range(1, num_qubits):
      for comb in itertools.combinations(qubit_index, k):
```

```
        isConnected = xor_bits_at_positions(result, comb, i)
        if isConnected:
          for edge in comb:
            graph.append({i, edge}) \\
        if {i, edge} not in graph else None

  return graph
```

Additionally, the following functions verify the condition $b_i \oplus \bigoplus_{j \in (i,j)} b_j = 0$ and generate the graph state from the given graph, represented by $U_{\text{gen}}$:

```
# Phase 3
def xor_bits_at_positions(nums, positions, base_location):
  for num in nums:
    # Start with 0 for XOR accumulation
    result = (num >> base_location) & 1

    # XOR each specified bit position
    for pos in positions:
      # Extract the bit at the current position
    # and XOR it with the result
      result ^= (num >> pos) & 1

    if result == 0:
      continue
    else:
      return False

  return True

# Given graph G genreate graph state.
def GraphState(num_qubits, graph):
  U_gen = qiskit.QuantumCircuit(num_qubits)
  for i in range(num_qubits):
    U_gen.h(i)

  for vertex in graph:
    temp = list(vertex)
    site_i = temp[0]
    site_j = temp[1]
    U_gen.cz(site_i, site_j)

  return U_gen
```

## Problem 3 : Low Depth Brickwork Circuits

The $N$-qubit pure state $|q_{N-1}q_{N-2}\cdots q_1q_0\rangle$ requires at least $2N+3$ additional circuits, as discussed in Problem 1, Part III. For $N > 10$, it is generally impossible to perform exact quantum state tomography (QST) under the problem's constraints: the limitations on the number of shots and circuits.

However, in some cases, if we have prior information about the state, we can perform a reasonably accurate QST with good approximations. Specifically, we know the circuit ansatz, which describes how the state is generated. The given 'one brickwork layer' circuit has low circuit depth(the longest path in the circuit is 2, with each brick contributing 1). This suggests that the generated state has low entanglement. In particular, the farther apart qubits $q_i$ and $q_j$ are, the less entanglement is expected between them.

Von Neumann entropy is a common measure of entanglement:

$$S = -\alpha_i \ln \alpha_i. \tag{116}$$

where $\alpha_i$ are Schmidt coefficients, ordered in descending order for convenience. To consider the state as less entangled, we assume that $|\alpha_0|^2 \gg |\alpha_i|^2$ for $i > 0$.

In this context, I propose the following three ideas:

**Idea 1.** From Schmidt decomposition, we have:

$$|q_{N-1}q_{N-2}\cdots q_1q_0\rangle = \sum_{k=0}^{3} \alpha_i \left|q_i^k q_{i+1}^k\right\rangle \left|\cdots q_{i-1}^k q_{i+2}^k \cdots\right\rangle \tag{117}$$

where $\left|q_i^k q_{i+1}^k\right\rangle$ and $\left|\cdots q_{i-1}^k q_{i+2}^k \cdots\right\rangle$ form orthonormal bases for each party. Assuming $|\alpha_0|^2 \gg |\alpha_k|^2$, we can approximate the state as:

$$|q_{N-1}q_{N-2}\cdots q_1q_0\rangle \simeq \left|q_{i+1}^0 q_i^0\right\rangle \left|\cdots q_{i+2}^0 q_{i-1}^0 \cdots\right\rangle \tag{118}$$

**Idea 2.** From Idea 1, we can approximate the 'one brickwork layer' circuit as a separable state, as shown in Eq. 118. Then, we can perform two-qubit QST to identify $\left|q_i^0 q_{i-1}^0\right\rangle$. For example, a POVM $\Omega$ acting on qubits $q_{i-1}$ and $q_i$ will give the estimated probability $m_\Omega$:

$$\begin{aligned} m_\Omega &= \langle q_{N-1}q_{N-2}\cdots q_1q_0| \,\Omega\, |q_{N-1}q_{N-2}\cdots q_1q_0\rangle \\ &\simeq \left\langle q_{i+1}^0 q_i^0\right| \Omega \left|q_{i+1}^0 q_i^0\right\rangle \left\langle \cdots q_{i+2}^0 q_{i-1}^0 \cdots \right| \left. \cdots q_{i+2}^0 q_{i-1}^0 \cdots \right\rangle \\ &= \left\langle q_{i+1}^0 q_i^0\right| \Omega \left|q_{i+1}^0 q_i^0\right\rangle \end{aligned} \tag{119}$$

Thus, we can find approximated state of $\left|q_i^0 q_{i-1}^0\right\rangle$.

**Idea 3.** In Eq. 118, we can perform one more Schmidt decomposition on the second party, consisting of $N-2$ qubits:

$$|q_{N-1}q_{N-2}\cdots q_1q_0\rangle \simeq \sum_{k=0}^{3} \beta_k \left|q^0_{j+1}q^0_j\right\rangle \left|q^0_{i+1}q^0_i\right\rangle \left|\cdots q^0_{j+2}q^0_{j-1}\cdots q^0_{i+2}q^0_{i-1}\cdots\right\rangle. \tag{120}$$

We can assume $\beta_0 = 1$. This approximation holds when the distance $|i-j| \geq 4$, since the circuit depth is 2. From Idea 2, we can now perform two-qubit QST on qubit pairs $(q_i, q_{i+1})$, $(q_j, q_{j+1})$, and more, simultaneously.

We can divide the 20 circuits into 4 phases, each consisting of 5 circuits. We need to select pairs of target qubits such that the condition $4 \leq |i-j|$ holds, where $q_i$ and $q_j$ are element of target qubit pairs. Below is the code I implemented to systematically choose the target qubit pairs for each phase.

```
def GenerateTargetQubitSet(num_qubits):
    conditions = [
        (0, 1),  # Phase 1: i % 4 == 0 or i % 4 == 1
        (2, 3),  # Phase 2: i % 4 == 2 or i % 4 == 3
        (1, 2),  # Phase 3: i % 4 == 1 or i % 4 == 2
        (3, 0)   # Phase 4: i % 4 == 3 or i % 4 == 0
    ]
    isTrue = lambda i,idx_condition :
              i % 4 == conditions[idx_condition][0]
              and (i+1) % 4 == conditions[idx_condition][1]
    target_qubit = [
      [(i, i + 1) for i in range(num_qubits - 1) if isTrue(i, idx)]
          for idx in range(4)
      ]
    return target_qubit
```

After determining the target qubit pairs for each phase, the 5 circuits for measurement in each phase are generated using the following code.

```
def GenerateMeasurementCircuits(U_hidden, target_set):
  hidden_circuit = U_hidden.copy()
  num_qubits = len(hidden_circuit.qubits)
  num_target = len(target_set)*2

  circuits = []
  phase_operations = {
    1: lambda qc, target: qc.h(target[0]),
```

```
    2: lambda qc, target: (qc.s(target[0]), qc.h(target[0])),
    3: lambda qc, target: qc.h(target[1]),
    4: lambda qc, target: (qc.s(target[1]), qc.h(target[1])),
  }

  for phase in range(5):
    qc = qiskit.QuantumCircuit(num_qubits, num_target)

    if phase in phase_operations:
      for target in target_set:
        phase_operations[phase](qc, target)

    # Add measurement
    for idx, target in enumerate(target_set):
      qc.measure(target[0], 2 * idx)
      qc.measure(target[1], 2 * idx + 1)

    result_circuit = hidden_circuit.compose(qc)

    circuits.append(result_circuit)

  return circuits
```

To perform QST on multiple qubit pairs, we need four detectors on qubits $q_i$, $q_{i+1}$, $q_j$, and $q_{j+1}$, resulting in $2^4 = 16$ possible measurement outcomes. Since we approximated the state as separable, we can combine measurement outcomes. For example, the measurement outcome $m_{0_i 0_{i+1}}$ can be calculated as:

$$m_{0_i 0_{i+1}} = m_{0_i 0_{i+1} 0_j 0_{j+1}} + m_{0_i 0_{i+1} 0_j 1_{j+1}} + m_{0_i 0_{i+1} 1_j 0_{j+1}} + m_{0_i 0_{i+1} 1_j 1_{j+1}} \quad (121)$$

Thus, we can perform two-qubit QST on multiple qubit pairs simultaneously, using only five circuits. By preparing four sets of qubit pairs, with each pair separated by at least 4 qubits, we can perform QST on all neighboring qubit pairs: $(q_0, q_1), (q_1, q_2), \cdots, (q_{N-2}, q_{N-1})$, by running 20 circuits. Below is the code that concatenates the measurement outcomes for a specific pair of qubits.

```
def SplitMeasurementResult(measurement_outcomes, target_set):
    # {(0,1) : [{"0x0" : ~ ,... }, {"0x0" : ~ ,... }, ...], (4,5): ~ , ...}]
    result = {}
    for idx, target in enumerate(target_set):
```

```
        target_result = []

        for measurement_outcome in measurement_outcomes:
            temp = {}
            for i in range(4):
                temp["0x" + format(i,"0x")] = 0
            for key, item in measurement_outcome.items():
                new_key_number = int(key,16) >> (2*idx) & 3
                new_key = "0x" + format(new_key_number,"0x")
                temp[new_key] += item
            target_result.append(temp)

        result[target] = target_result

    return result
```

After extracting the two-qubit states for all pairs $(q_0, q_1), (q_1, q_2), \cdots, (q_{N-2}, q_{N-1})$, then I reconstructed the circuit. The first layer, with qubit pairs $(q_0, q_1), (q_2, q_3), \cdots, (q_{2k}, q_{2k+1})$ is straightforward to reconstruct, as done in Problem 1, Part II. To reconstruct the second layer, I applied a trick using Schmidt decomposition for the state of the qubits $q_i, q_{i+1}, q_{i+2}, q_{i+3}$. The state can be expressed as:

$$|q_{i+3}q_{i+2}q_{i+1}q_{i+1}\rangle \simeq |q_{i+2}q_{i+1}\rangle\,|q_{i+3}q_i\rangle \tag{122}$$

$$= \sum_{k=0}^{3} \lambda_0 \left|q_{i+2}^k q_{i+3}^k\right\rangle \left|q_i^k q_i^k\right\rangle \tag{123}$$

From the first row, I performed Schmidt decomposition to arrive at the second row. Next, I implemented the unitary operation to map: $\left|q_{i+2}^0 q_{i+3}^0\right\rangle \to |q_{i+2}q_{i+3}\rangle$. Such process is implemented in the following code.

```
def SecondLayerCircuit(target_state, neighbor_state):
  if len(neighbor_state) == 2:
    state1 = neighbor_state[0]; state2 = neighbor_state[1]
    a_jm_in = np.einsum("ij,mn->jmin",state2, state1)
                  .reshape(4,4) # jm x in

    u, d, v = np.linalg.svd(a_jm_in)
    e = u[:,0]

    # e -> |00> -> target state
    U_d = TwoQubitCircuit(e)
    U = TwoQubitCircuit(target_state)
```

```
    U_gen = (U_d.inverse()).compose(U)

    return U_gen
  else:
    state1 = neighbor_state[0];
      state2 = np.array([1,0], dtype=np.complex64)
    a_k_ij = np.einsum("k,ij->kij",state2, state1)
                 .reshape(4,2) # ki x j

    u, d, v = np.linalg.svd(a_k_ij)
    e = u[:,0]

    U_d = TwoQubitCircuit(e)
    U = TwoQubitCircuit(target_state)

    U_gen = (U_d.inverse()).compose(U)

    return U_gen
```

## References


[1] Adriano Barenco, Charles H. Bennett, Richard Cleve, David P. DiVincenzo, Norman Margolus, Peter Shor, Tycho Sleator, John A. Smolin, and Harald Weinfurter. Elementary gates for quantum computation. *Phys. Rev. A*, 52:3457–3467, Nov 1995.

[2] Raban Iten, Roger Colbeck, Ivan Kukuljan, Jonathan Home, and Matthias Christandl. Quantum circuits for isometries. *Phys. Rev. A*, 93:032318, Mar 2016.

[3] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.

[4] Oscar Perdomo, Nelson Castaneda, and Roger Vogeler. Preparation of 3-qubit states, 2022.

[5] Martin Plesch and Časlav Brukner. Quantum-state preparation with universal gate decompositions. *Phys. Rev. A*, 83:032302, Mar 2011.